

Tenstorrent CCL 지원 현황 및 코드 구현 상세 내용

Tenstorrent NPU

2025.04.07 한국항공대학교 이유찬

Contents

- ▶ TT-NN CCL components
 - ▶ EDM (ERISC Data Mover) Engine
 - ▶ Topology
- ▶ Samples
 - ▶ Linear all-gather
 - ▶ Ring all-gather

TT-NN CCL components

- ▶ EDM Builder
 - ▶ Worker와 EDM 간 커널 인수 간소화
 - ▶ EDM arguments 추가 간소화
 - ▶ Buffering과 EDM 메모리 관리
- ▶ Tensor Slicer(Scheduler)
 - ▶ 각 worker가 작업에 할당된 tensor slice를 생성하도록 도움
- ▶ Topology Config
 - ▶ Ring, Line topology 구성

TT-NN CCL components

EDM Engine

EDM(ERISC Data Mover) Engine

- ▶ Ethernet 링크를 통한 데이터 전송을 위한 구성요소
- ▶ ERISC core에서 동작하는 SW component로 구현됨
- ▶ Multichip 집합 작업 고려사항을 처리하고 Ethernet 활용률을 제공하기 위해 디자인 됨
- ▶ Multi-channel, bidirectional 데이터 전송 지원
- ▶ CCL(Collective Communication Library) 작업 수행 가능

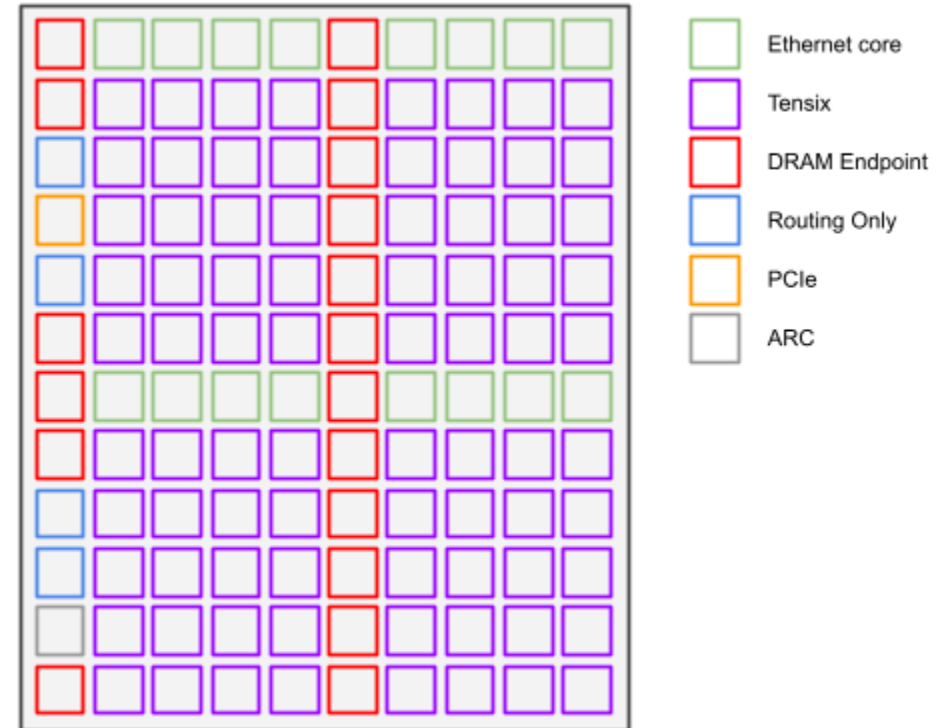


사진 출처: tt-metal github tech_reports

EDM Communication Interface

▶ Signaling

- ▶ Ethernet link 사용 가능 여부 확인을 위해 본 작업 수행 전 Handshake 수행
- ▶ sender와 receiver 지정

▶ Termination

- ▶ 2가지 모드 존재
- ▶ Fixed message count
- ▶ Worker driven termination signal
- ▶ CCL은 작업에 따라 두 방식을 복합적으로 사용
- ▶ [future work] All-gather는 signal termination mode 사용 예정

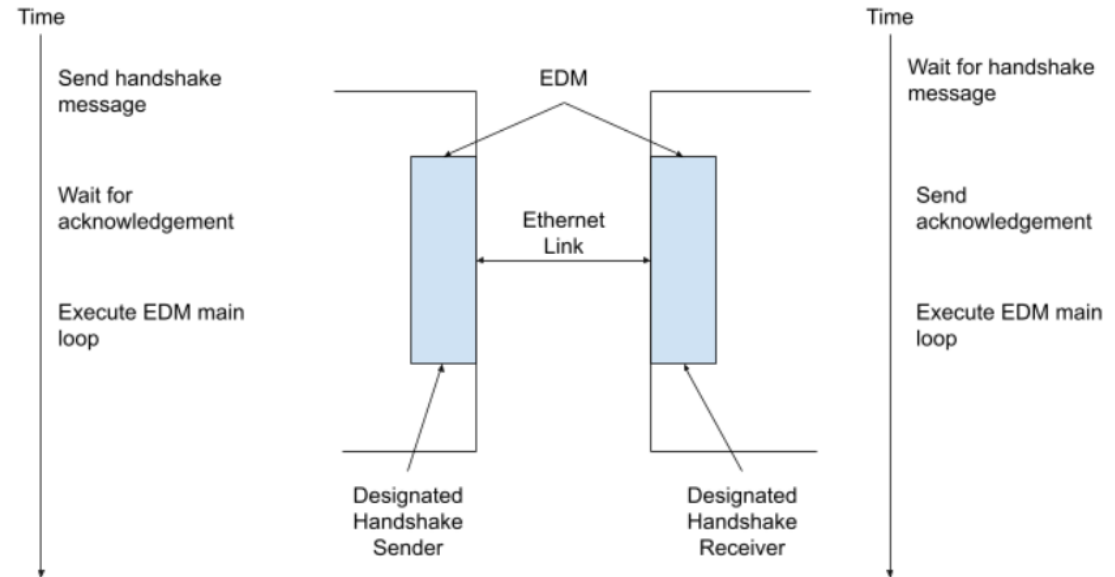


사진 출처: tt-metal github tech_reports

Worker-Sender Side EDM Interface

- ▶ ERISC와 worker 간 signaling은 semaphore 한 쌍을 통해 수행
- ▶ EDM sender에 channel buffer 위치(sender worker core와 공유)
- ▶ EDM semaphore
 - ▶ Worker에 의해 증가
 - ▶ Worker가 EDM에 새로운 payload 생겼음을 알림
- ▶ Sender semaphore
 - ▶ ERISC에 의해 증가
 - ▶ 워커가 sender channel buffer 사용할 수 있음을 알림

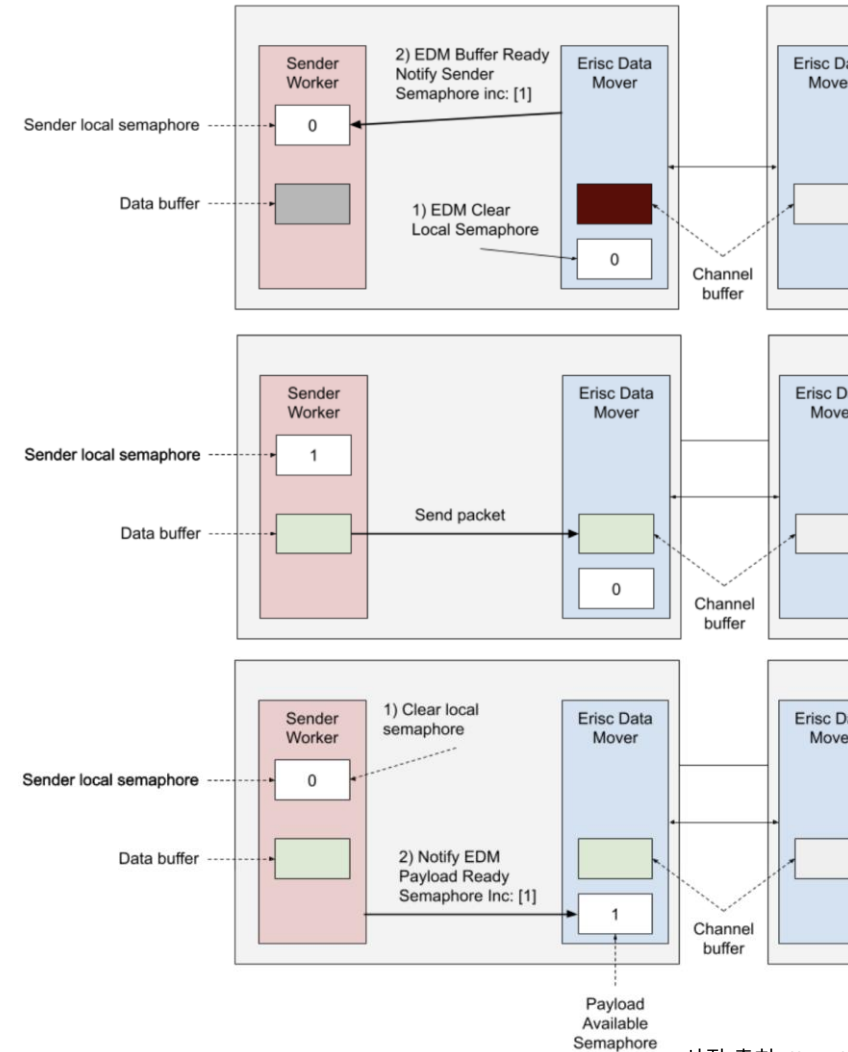


사진 출처: tt-metal github tech_reports

Receiver Side EDM-Worker Interface

- ▶ EDM receiver channel과 receiver worker는 유사한 semaphore와 channel buffer 가짐
 - ▶ 차이점은 Worker가 데이터를 receiver로 부터 pull하는 방식
- ▶ Semaphore는 payload 사용 가능과 읽음 상태 확인용
- ▶ [Future work] push 방식 추가 예정

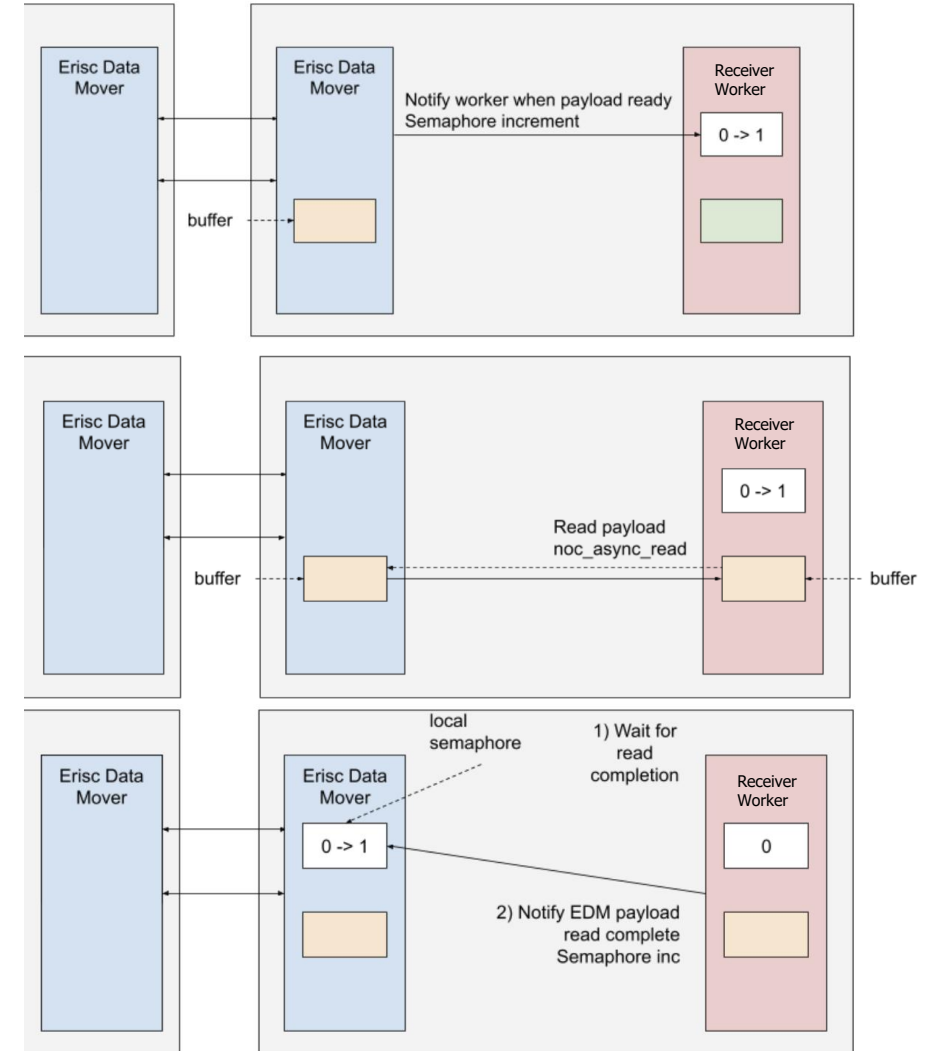


사진 출처: tt-metal github tech_reports

TT-NN CCL components

Topology

Op List

▶ 현재 지원되는 CCL ops

2024 7월 기준

▪ CCL

▪ `ttnn.all_gather`

- `all_gather()`

▪ `ttnn.reduce_scatter`

- `reduce_scatter()`

▪ `ttnn.experimental.all_reduce`

- `all_reduce()`

Configuration/Layout	All Gather	Reduce Scatter	Send/Receive	All Reduce
Interleaved (Row Major)	Y	N	N	N
Interleaved (Tile)	Y	Y	N	N
Width Sharded (Tile)	Y	Y	N	N
Width Sharded (Row Major)	Y	N	N	N
Height Sharded (Tile)	Y	N	N	N
Height Sharded (Row Major)	N	N	N	N
Block Sharded (Tile)	Y	Y	N	N
Block Sharded (Row Major)	N	N	N	N
Bidirectional	Y	N	N	N

(Ring) All-gather

- ▶ 모든 worker들은 2core에 걸쳐 동작
 - ▶ EDM으로 부터 수신, EDM으로 발신
- ▶ Sender worker는 input tensor를 output tensor와 EDM으로 전송
- ▶ Receiver worker는 EDM으로 부터 수신된 payload를 해당하는 Output Tensor t 위치에 채워 넣음
- ▶ Sender worker는 local semaphore가 증가하면 output tensor를 전송

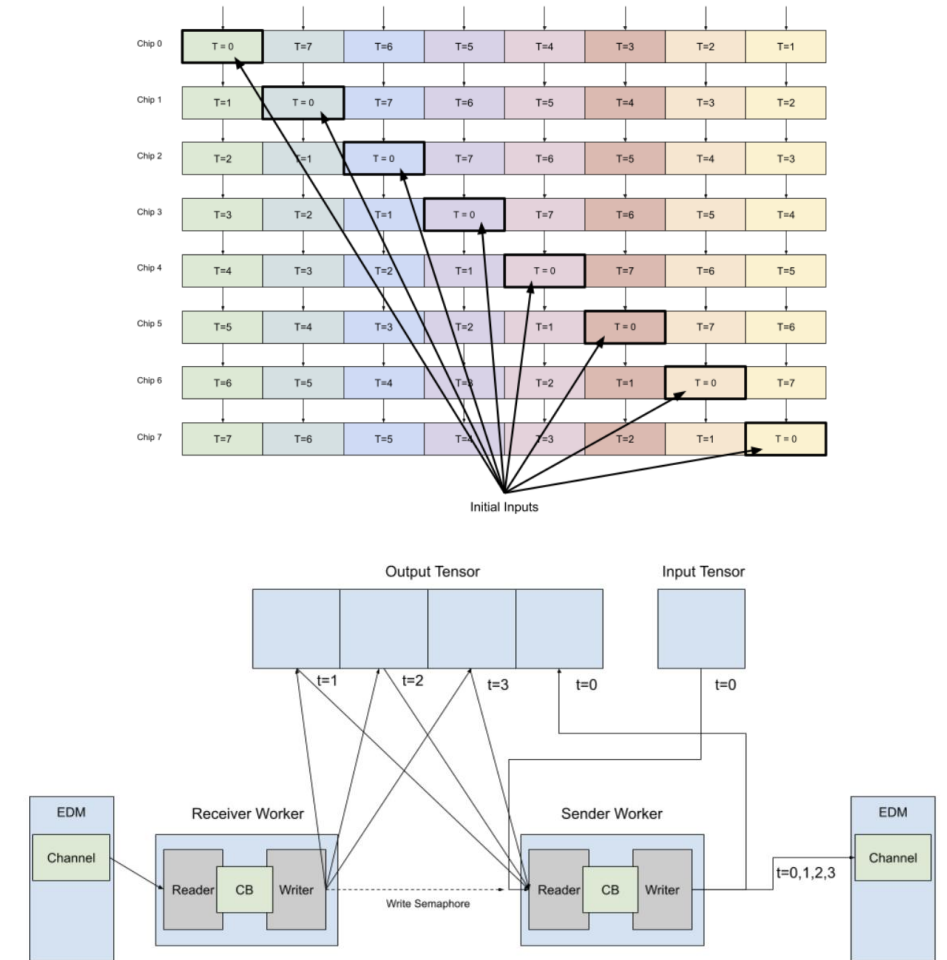


사진 출처: tt-metal github tech_reports

Bidirectionality

- ▶ ERISC는 Bidirectional link를 가짐
 - ▶ 각 link(단방향) 당 100Gbps
 - ▶ 양방향 link 200Gbps
- ▶ 양방향의 bandwidth를 모두 활용하기 위해 Bidirectionality 지원
- ▶ Ring: Tensor의 절반을 반대 방향으로 전송
- ▶ Line: 본질적으로 양방향 동작
 - ▶ 대역폭 사용 특성차이로 Ring에 비해 성능 저하 발생

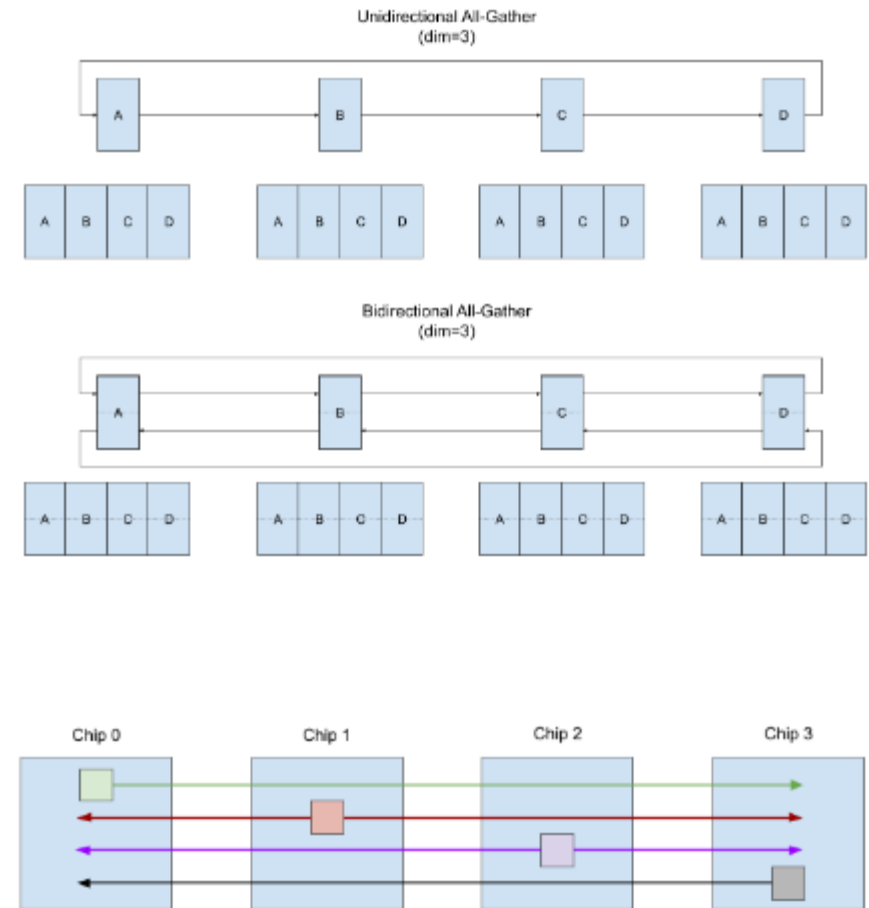


사진 출처: tt-metal github tech_reports

(Ring) Reduce Scatter

- ▶ All-gather와 달리 추가적인 작업이 요구됨
- ▶ 필요에 따라 tensor를 더 작게 나누어 수행하는 경우도 존재
 - ▶ 여러 tensor로 나누어 실행되더라도 하나의 호출로 실행

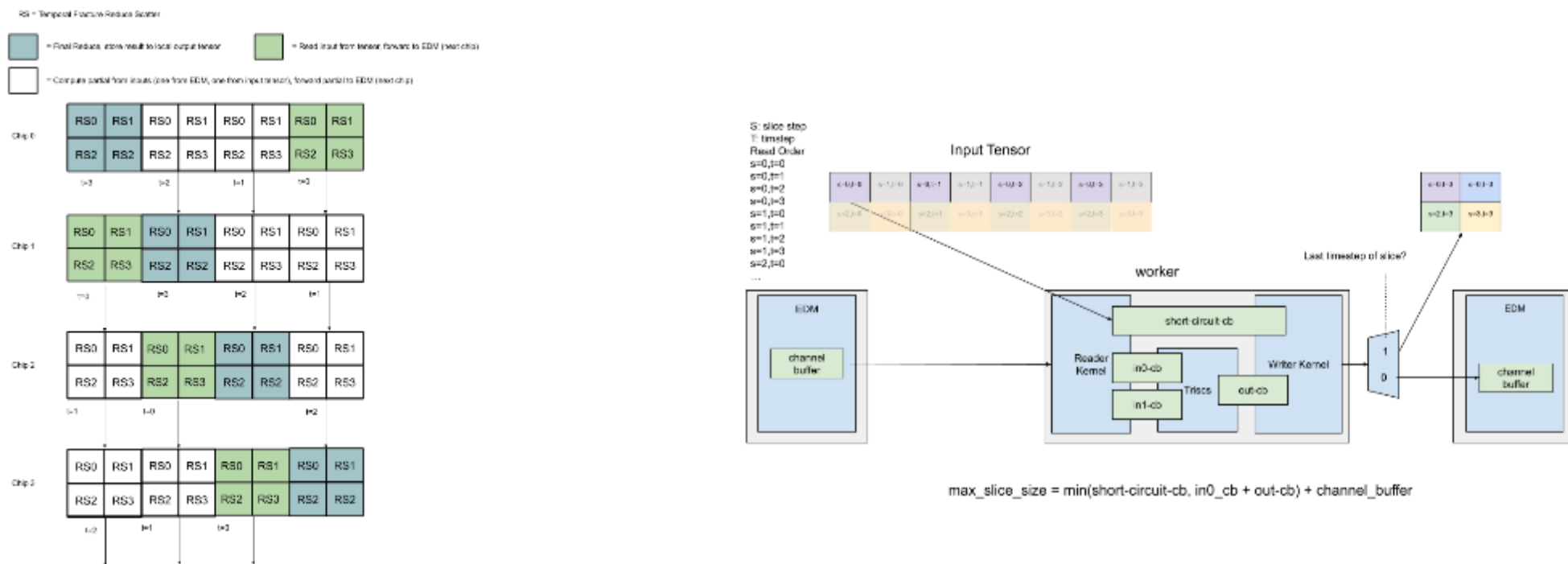


사진 출처: tt-metal github tech_reports

TT-NN CCL

Samples

Line All-gather 2X4

▶ 2X4 topology에서 axis0에 대한 line all-gather 수행

```
>>> mesh_tensor = ttnn.from_torch(
...     torch_tensor,
...     layout=ttnn.TILE_LAYOUT,
...     device=mesh_device,
...     mesh_mapper=ttnn.ShardTensorToMesh(mesh_device, dim=3),
... )
>>> ttnn.visualize_mesh_device(mesh_device, tensor=mesh_tensor)
MeshDevice(rows=2, cols=4):
```

Dev. ID: 4 (0, 0) Shape([1, 1, 32, 32])	Dev. ID: 0 (0, 1) Shape([1, 1, 32, 32])	Dev. ID: 3 (0, 2) Shape([1, 1, 32, 32])	Dev. ID: 7 (0, 3) Shape([1, 1, 32, 32])
Dev. ID: 5 (1, 0) Shape([1, 1, 32, 32])	Dev. ID: 1 (1, 1) Shape([1, 1, 32, 32])	Dev. ID: 2 (1, 2) Shape([1, 1, 32, 32])	Dev. ID: 6 (1, 3) Shape([1, 1, 32, 32])

```
>>> output_tensor = ttnn.all_gather(mesh_tensor, dim=3, cluster_axis=0, mesh_device=mesh_device, topology=ttnn.Topology.Linear,)
>>> ttnn.visualize_mesh_device(mesh_device, tensor=output_tensor)
MeshDevice(rows=2, cols=4):
```

Dev. ID: 4 (0, 0) Shape([1, 1, 32, 64])	Dev. ID: 0 (0, 1) Shape([1, 1, 32, 64])	Dev. ID: 3 (0, 2) Shape([1, 1, 32, 64])	Dev. ID: 7 (0, 3) Shape([1, 1, 32, 64])
Dev. ID: 5 (1, 0) Shape([1, 1, 32, 64])	Dev. ID: 1 (1, 1) Shape([1, 1, 32, 64])	Dev. ID: 2 (1, 2) Shape([1, 1, 32, 64])	Dev. ID: 6 (1, 3) Shape([1, 1, 32, 64])

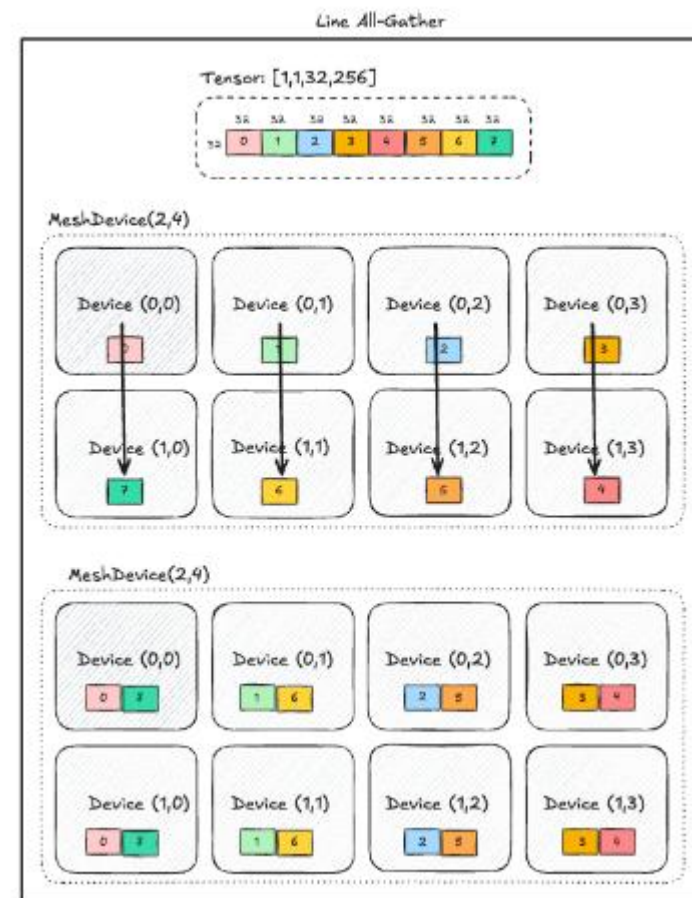


사진 출처: tt-metal github tech_reports

Ring All-gather 2X4

▶ 2X4 topology에서 전체 노드에 대한 ring all-gather 수행

```
>>> output_tensor = tttn.all_gather(mesh_tensor, dim=3, num_links=1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/data/yuchan/tt-metal/ttnn/ttnn/decorators.py", line 333, in __call__
    return self.function(*function_args, **function_kwargs)
RuntimeError: TT_FATAL @ /data/yuchan/tt-metal/tt-metal/llrt/tt_cluster.cpp:1076: local_ethernet_sockets.find(remote_chip) != local_ethernet_sockets.end()
info:
Device 4 is not connected to Device 6
```

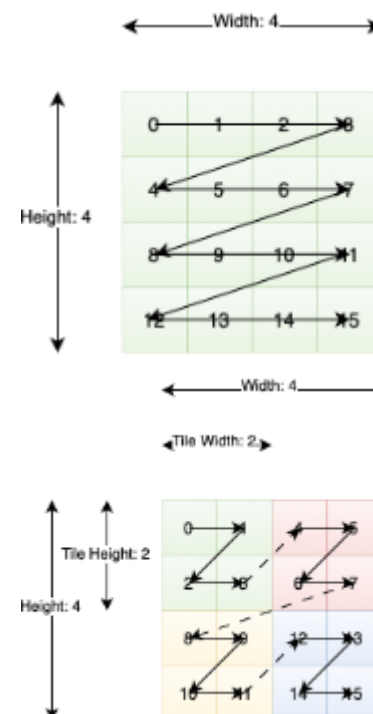
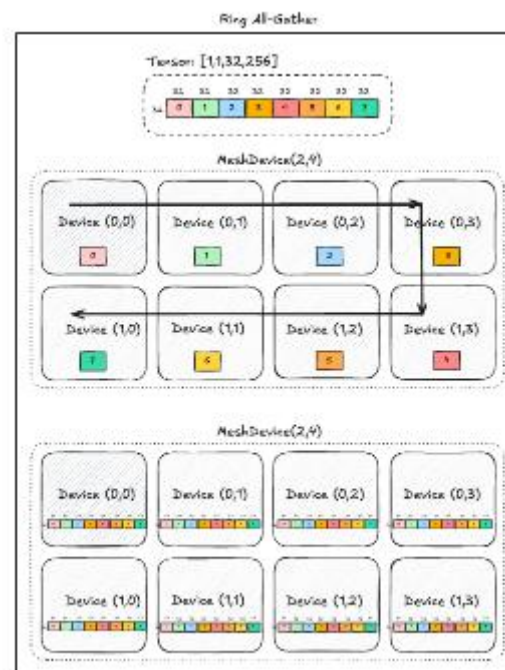
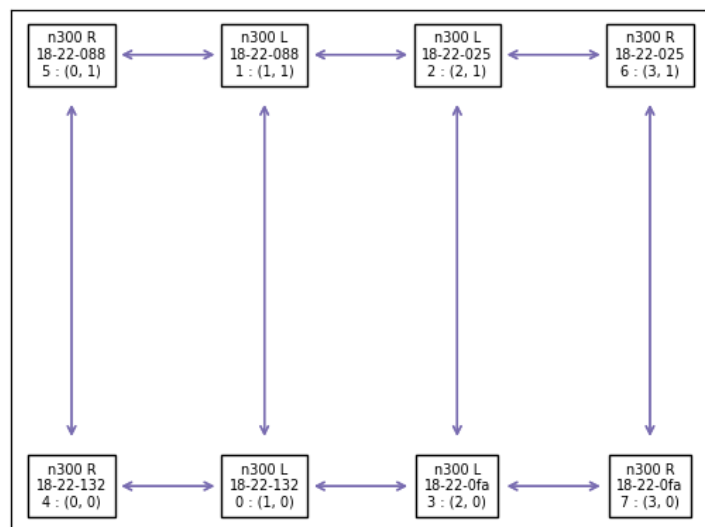


사진 출처: tt-metal github tech_reports, tt-metal docs

Ring All-gather topology

▶ 1X8 topology ring all-gather 수행

```
>>> ttnn.visualize_mesh_device(mesh_device, tensor=mesh_tensor)
MeshDevice(rows=1, cols=8):
```

Dev. ID: 4 (0, 0) Shape([1, 1, 32, 32])	Dev. ID: 0 (0, 1) Shape([1, 1, 32, 32])	Dev. ID: 3 (0, 2) Shape([1, 1, 32, 32])	Dev. ID: 7 (0, 3) Shape([1, 1, 32, 32])	Dev. ID: 6 (0, 4) Shape([1, 1, 32, 32])	Dev. ID: 2 (0, 5) Shape([1, 1, 32, 32])	Dev. ID: 1 (0, 6) Shape([1, 1, 32, 32])	Dev. ID: 5 (0, 7) Shape([1, 1, 32, 32])
---	---	---	---	---	---	---	---

```
>>> output_tensor = ttnn.all_gather(mesh_tensor, dim=3, num_links=1)
>>> ttnn.visualize_mesh_device(mesh_device, tensor=output_tensor)
MeshDevice(rows=1, cols=8):
```

Dev. ID: 4 (0, 0) Shape([1, 1, 32, 256])	Dev. ID: 0 (0, 1) Shape([1, 1, 32, 256])	Dev. ID: 3 (0, 2) Shape([1, 1, 32, 256])	Dev. ID: 7 (0, 3) Shape([1, 1, 32, 256])	Dev. ID: 6 (0, 4) Shape([1, 1, 32, 256])	Dev. ID: 2 (0, 5) Shape([1, 1, 32, 256])	Dev. ID: 1 (0, 6) Shape([1, 1, 32, 256])	Dev. ID: 5 (0, 7) Shape([1, 1, 32, 256])
--	--	--	--	--	--	--	--

▶ 2X2 topology ring all-gather 수행

▶ 2X4 topology와 동일한 문제 발생

```
>>> ttnn.visualize_mesh_device(mesh_device, tensor=mesh_tensor)
MeshDevice(rows=2, cols=2):
```

Dev. ID: 4 (0, 0) Shape([1, 1, 32, 32])	Dev. ID: 0 (0, 1) Shape([1, 1, 32, 32])
Dev. ID: 5 (1, 0) Shape([1, 1, 32, 32])	Dev. ID: 1 (1, 1) Shape([1, 1, 32, 32])

```
>>> output_tensor = ttnn.all_gather(mesh_tensor, dim=3, num_links=1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/data/yuchan/tt-metal/ttnn/ttnn/decorators.py", line 333, in __call__
    return self.function(*function_args, **function_kwargs)
RuntimeError: TT_FATAL @ /data/yuchan/tt-metal/tt-metal/llrt/tt_cluster.cpp:1076: local_ethernet_sockets.find(remote_chip) != local_ethernet_sockets.end()
info:
Device 4 is not connected to Device 1
```